

Step 3 - Formats & Partitioning

Phase A · CSV to Parquet to Delta · Pruning · Enforcement · ~20 min

Goal: Convert CSV to Parquet and Delta, partition by date, prove partition pruning with explain(), and watch Delta schema enforcement reject a bad write. Artifact: 03_formats_partitioning.ipynb

LEARN

Format	Nature	Banking role
CSV	Row-oriented, no schema, no compression	Inbound from legacy sources; outbound to regulators (FINTRAC requires it)
JSON	Nested, verbose	Interac feeds, API payloads
Parquet	Columnar, compressed, embedded schema, predicate pushdown	Lake storage workhorse
Delta	Parquet + transaction log	Bronze onward: ACID, time travel, schema enforcement

Partitioning = folders by column value. Filter on the partition column and Spark skips entire folders (partition pruning): 1 day read instead of 1,095.

Lifecycle rule: sources arrive CSV/JSON; Bronze onward is Delta; regulatory outputs go back out as CSV.

LAB — notebook 03_formats_partitioning (Serverless)

Cell 1: reload df_txn with explicit schema (as Step 1). Then:

Cell 2 — Write three ways:

```
df_txn.write.mode("overwrite").partitionBy("transaction_date") \
    .parquet(f"{VOL}/formats_lab/txn_parquet")

df_txn.write.format("delta").mode("overwrite").partitionBy("transaction_date") \
    .save(f"{VOL}/formats_lab/txn_delta")
```

Cell 3 — See the folder-per-date structure:

```
for f in dbutils.fs.ls(f"{VOL}/formats_lab/txn_parquet")[:8]:
    print(f.name)
# transaction_date=2025-11-01/, transaction_date=2025-11-02/, ...
```

Cell 4 — Size comparison:

```
def total_size(path):
    size = 0
    for f in dbutils.fs.ls(path):
        size += total_size(f.path) if f.isDir() else f.size
    return size

csv_size = [f.size for f in dbutils.fs.ls(VOL) if f.name == "fact_transactions.csv"][0]
pq_size = total_size(f"{VOL}/formats_lab/txn_parquet")
print(f"CSV {csv_size/1024:.0f} KB Parquet {pq_size/1024:.0f} KB ratio {csv_size/pq_size:.1f}x")
```

On 10K rows expect a modest 2-4x (per-file overhead dominates). At bank scale it reaches 5-10x. Knowing WHY the small ratio happens is itself an interview point.

Cell 5 — Partition pruning, proven:

```
df_pruned = spark.read.format("delta").load(f"{VOL}/formats_lab/txn_delta") \
    .filter(F.col("transaction_date") == "2025-11-15")
print(df_pruned.count())
df_pruned.explain() # find PartitionFilters: [...transaction_date...]
```

Cell 6 — Schema enforcement (Delta protects you):

```
df_bad = df_txn.limit(10).withColumn("surprise_column", F.lit("oops"))
try:
    df_bad.write.format("delta").mode("append").save(f"{VOL}/formats_lab/txn_delta")
    print("should NOT succeed")
except Exception as e:
    print("Delta REJECTED the write - enforcement works:", type(e).__name__)
```

Cell 7 — Schema evolution (opt-in change):

```
df_bad.write.format("delta").mode("append") \
    .option("mergeSchema", "true").save(f"{VOL}/formats_lab/txn_delta")
# new column added; old rows have NULL for it
```

Cell 8 — Reset to clean 10,000 rows:

```
df_txn.write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \
    .partitionBy("transaction_date").save(f"{VOL}/formats_lab/txn_delta")
```

COMMIT + CHECKPOINT

Export to pyspark/. Pass criteria: PartitionFilters visible in explain(); enforcement error caught in Cell 6; mergeSchema append succeeds in Cell 7. Production rule: mergeSchema stays FALSE by default; enabling it requires an approved change request - upstream surprises must never silently alter the data contract.